

Data Cleansing & Train/Test Split

- Course: Machine Learning
- Lecture Topic: Data Preparation for ML

Learning Objectives

- Understand data cleansing
- Apply cleaning techniques
- Understand train/test split

Why Data Cleansing Matters

- Real-world data is messy
- Dirty data leads to wrong insights
- ML models depend on data quality
- If data is bad, the model will also be bad.

Common Data Quality Issues

- **Missing values** – empty cells. Age not entered, Sensor failed, Survey question skipped
- **Duplicates** – duplicate rows. Same person entered two times
- **Incorrect data** - Salary = “abc”
- **Inconsistent data** - “Male”, “male”, “M”, “Finland”, “FIN”, “Suomi”

- **Outliers** - Values that are very different from most data:
- Salary = 1,000,000 while others earn 3,000

Data Profiling (always first thing)

- Before cleaning, **look at the data.**
- `data.head()` – shows first 5 rows
- `data.info()`- shows data types of columns, missing values
- `data.describe()`- Mean, min, max (numbers)
- `data.isna().sum()` - Shows how many values are missing (not available)

Handling Missing Values

- **Option 1: Drop**
- **`data.dropna()`**
- Remove rows or columns
- Use Only if very few missing
- **Option 2: Fill (Imputation)**
- **`data.fillna()`**
- Numbers → mean or median
- Text → most common value (mode)

Fixing Data Types

- Convert strings to numbers
 - `.to_numeric()` function
- Convert to date type
 - `.to_datetime()`
- - Use `errors='coerce'`

Making text consistent

- `.str.lower()`
- `.replace({})`

Removing Duplicates

- `data.duplicated()`
- `data.drop_duplicates()`

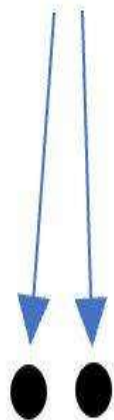
Handling Outliers

- Easiest way to identify outliers
- - using seaborn library boxplot
import seaborn as sns
sns.boxplot(x=data['columnname'])

IQR method(standard ML approach)

- IQR = Interquartile range
- Removes extreme values that are far from the normal range
- Calculating quantiles
- E.g calculating the value below which 25% of values fall (q_1)
- Calculating the value below which 75% of values fall (q_2)
- $Iqr = q_2 - q_1$ (middle range)

Outliers



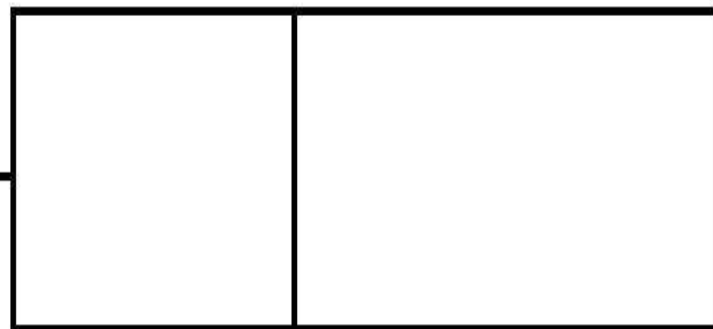
Min

$(Q1 - 1.5 * IQR)$

Q1

Median

Q3



Max

$(Q3 + 1.5 * IQR)$

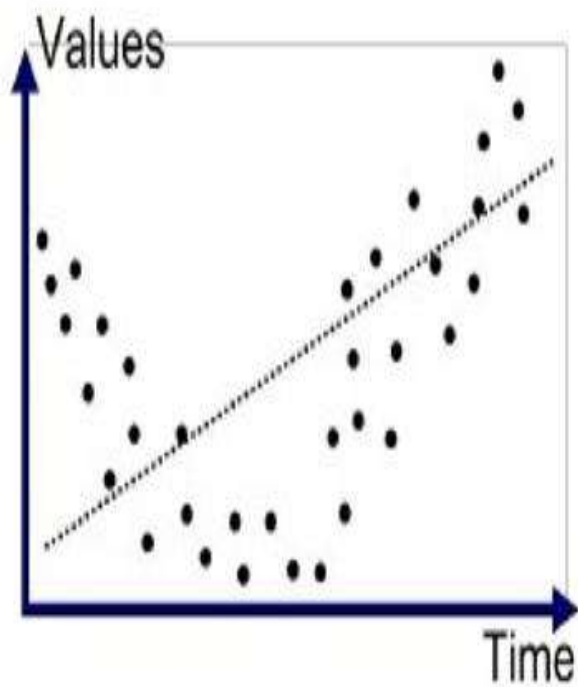
Outlier



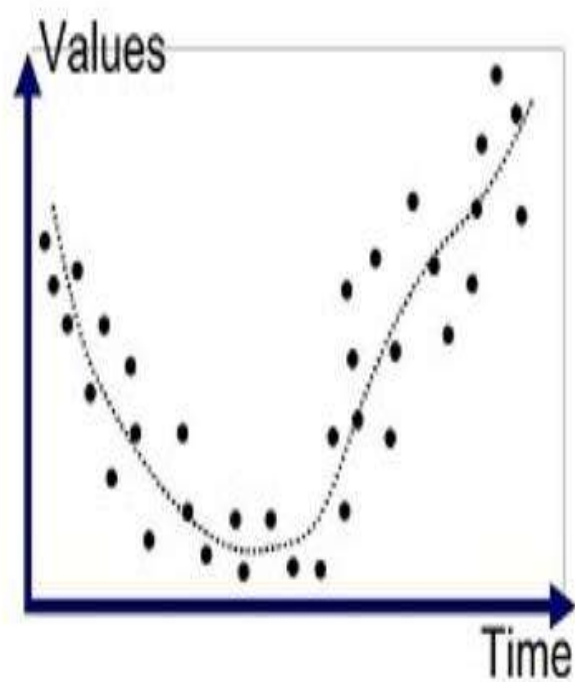
- Detect outliers
- Not every outlier is wrong (e.g salary of CEO)
- Decide using **context**
- Do NOT blindly delete

Why Train/Test Split

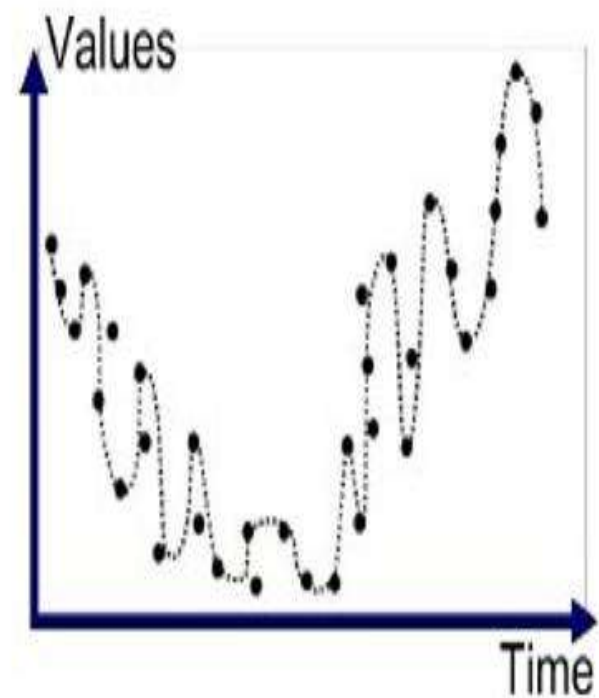
- If we train and test on same data:
- Model memorizes answers
- Accuracy looks very high
- But fails on new data
- This is called **overfitting**.



Underfitted



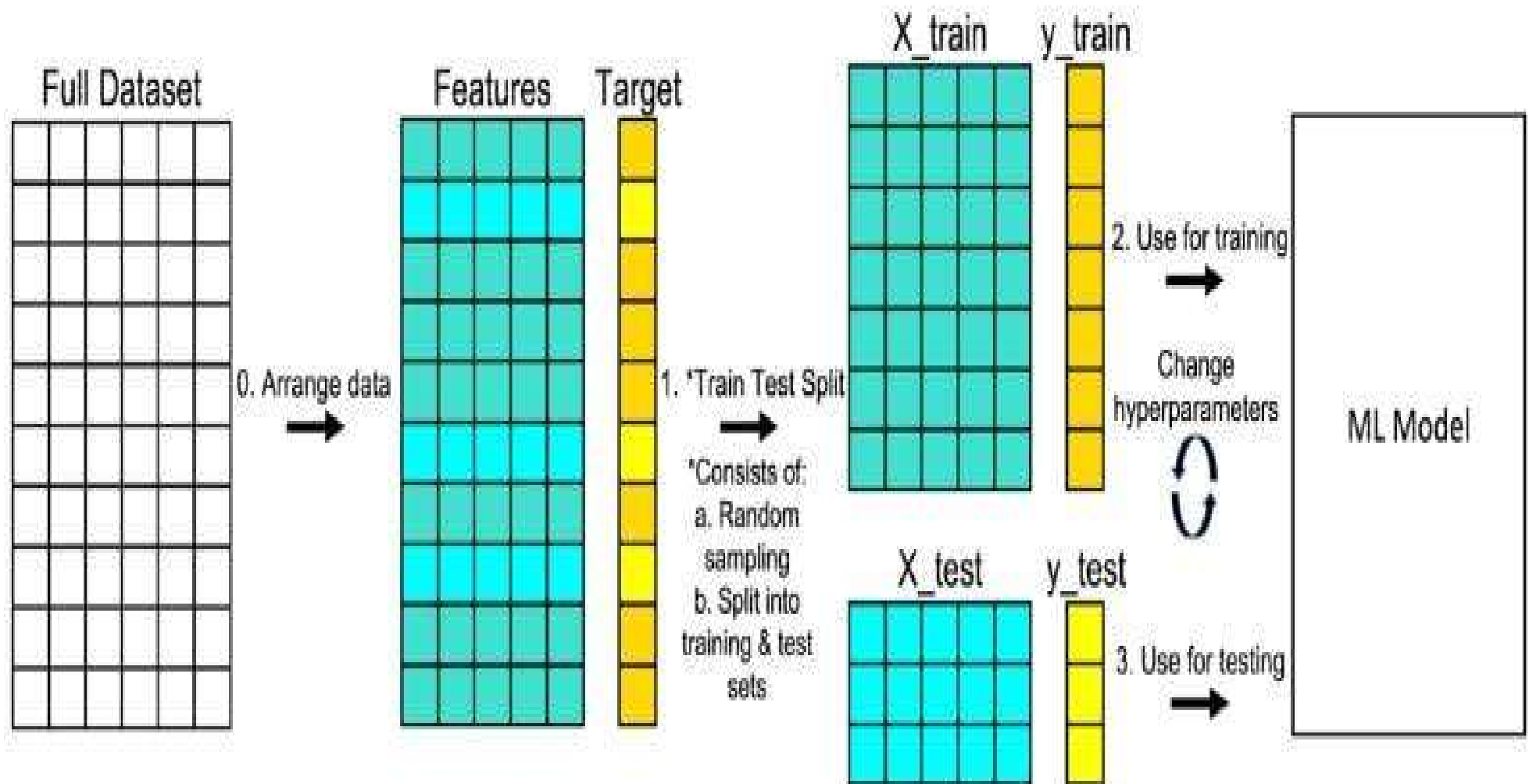
Good Fit/Robust



Overfitted

Train/Test Split Concept

- Training set: learn
- Test set: evaluate
- Typical split: 80/20



Train/Test Split in Python

- `train_test_split(X, y, test_size=0.2, random_state=42)`
- Test size 0.2 - means (20% test)

Stratified Split

- Stratified splitting ensures that train and test sets have the same class proportions as the original dataset.
- i.e each class is represented fairly in both training and testing data
- Keeps class balance

- Imagine this dataset:
- 90% **Not Spam**
- 10% **Spam**
- If we split **randomly**, the test set might end up with:
 - 0 spam emails or too many spam emails
 - Accuracy becomes misleading
 - Model evaluation is unfair

- `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`
- `stratify=y` → preserves class distribution of `y`

Iris Dataset – What Are We Classifying?

- **Dataset:** Iris flower dataset (classic machine learning dataset)
- **Goal (Classification Task):**
Predict the species of a flower based on its measurements

- **Features (X) – What we measure**
- Each flower has 4 numeric features:
- Sepal length
- Sepal width
- Petal length
- Petal width
- Each row = one flower

- **Target (y) – What we predict**
- Flower species (class label):
- Setosa
- Versicolor
- Virginica
- (Encoded as 0, 1, 2 in the data)